Concepts Shared by All Predictors: Colloquium Textbook

Thesis: A Unified Benchmark of Grokking Predictors in Neural Networks · Common concepts (not predictor-specific) · Prepared 5 November 2026

This is the study book for the concepts shared across your whole benchmark: 7 modules and 29 topics. Every term is defined in the Notation and glossary chapter, before it’s used.

Jonathan John · M.Sc. AI

Contents

Notation and glossary **2**

1 The grokking phenomenon **8**

1.1 Grokking 8

1.2 Memorisation vs generalisation 8

1.3 Circuit formation and cleanup 9

2 Your training setup **10**

2.1 Modular addition task 10

2.2 Train/test split (30%) 11

2.3 Transformer, logits and cross-entropy loss 12

2.4 AdamW and weight decay 14

2.5 Epochs and checkpoints 15

2.6 Seeds and why five 16

3 Measuring the outcome **18**

3.1 Test accuracy 18

3.2 The grok epoch G_i 19

4 What a predictor is **20**

4.1 Predictor and event epoch P_i 20

4.2 Lead and the 100-epoch margin 20

4.3 Lead fraction 21

4.4 Relative lead and mean relative lead 22

4.5 Misses and `None` 24

5 Spearman's ρ and small samples **26**

5.1 Spearman rank correlation ρ 26

5.2 Pearson vs Spearman 27

5.3 Only 21 possible values with 5 seeds 29

5.4 Significance with small n 30

6 The frozen protocol and the PASS rule **32**

6.1 The frozen protocol 32

6.2 The PASS rule 32

6.3 Pre-registration and criterion validation 34

7 Research honesty **36**

7.1 Hindsight (look-ahead) bias 36

7.2 Causal vs centred moving average 37

7.3 Researcher degrees of freedom 38

7.4 Sensitivity analysis 38

7.5 Baselines, controls and noise floor 39

7.6 Negative results and reproducibility 39

Recommended reading **40**

Notation and glossary

Every symbol and term in this book is defined here, once, in dependency order: each entry uses only entries above it. The modules use only these names. Entries are numbered **A1**, **N1**, **N2**, … so the text can point back to them. For each entry you get what it is, its type, where it's evaluated, its sign or direction where that matters, and its role in the benchmark.

Part A. How to read the maths

A1 Set, $\in$, #. A set is a collection of objects written in braces, e.g. $\{0, 1, 2\}$. $x \in S$ means "x belongs to the set S". $\#S$ is an integer: the number of objects in S.

A2 $\min\{\ldots\}$**.** The smallest number in a set of numbers.

A3 $\lfloor x \rfloor$ **(floor).** For a scalar $x \geq 0$, the largest integer that is $\leq x$. Python's `int()` gives the same result for $x \geq 0$.

A4 mod. For whole numbers $x \geq 0$ and $y > 0$, x mod y is the remainder when x is divided by y: a whole number from 0 to $y - 1$. For example, 150 mod $113 = 37$. (x and y are placeholders in this entry only.)

A5 $=$ **and** $\approx$**.** $=$ means exactly equal. $\approx$ means approximately equal. In a derivation $\approx$ appears once, on the line where digits or terms are dropped, and that line says what was dropped.

A6 $\because$ **and** $\therefore$**.** $\because$ means "because"; every line of a solution ends with one. $\therefore$ means "therefore"; it opens the conclusion.

A7 Derivation layout (H.K. Dass). *Given*: a numbered list of facts, each naming where it comes from (a glossary entry, a section, or a file of yours). *Let*: each symbol used, defined once. *To find*: what is computed or shown. *Solution*: one aligned block, one step per line. Then a $\therefore$ sentence in plain words and a boxed answer.

A8 d.p. Decimal places. "0.991 (3 d.p.)" means the value rounded to 3 decimal places.

A9 Your files. `nanda_unified.yaml`: the configuration (config) file of the final setup. `analyze_nanda_unified.py`: the analysis script. `summary.json`: the saved results file. `context.md`: your project log.

A10 Citations [1]–[6]. Numbers in square brackets point to the Recommended reading list at the end. "Power et al." is [1]; "Nanda et al." or "Nanda" is [2].

A11 Viva, examiner, attack point. The viva is your oral exam (colloquium). The examiner asks the questions. An attack point is a weakness in your work that an examiner could raise there. An audit finding is an issue found in the 3 November 2026 audit of your code. Your viva notes are the answers you prepared for the viva.

Part B. The task and the data

N1 p**, the modulus.** Integer, prime, $p = 113$. Set in `nanda_unified.yaml` and never changed. Role: sets the size of the task.

N2 a, b**, the inputs.** Two integers, each in $\{0, 1, \ldots, p-1\} = \{0, \ldots, 112\}$. Role: the two numbers the model reads.

N3 c**, the correct answer.** Integer, $c = (a + b) \bmod p$, so $c \in \{0, \ldots, 112\}$. Role: the label the model must output.

N4 Pair and N**.** A pair is one example: the inputs (a, b) together with their answer c. N is an integer, the number of all pairs: $N = p^2 = 12{,}769$ (Section 2.1).

N5 The model, its weights θ**, and training.** The model is a one-layer, four-head transformer (a standard type of neural network; "one-layer, four-head" names its size, and its insides aren't needed here). It's a function: it reads a pair's inputs (a, b) and a weight vector θ, and returns 113 scores, one per possible answer. θ is a vector with M entries $\theta_1, \ldots, \theta_M$: every trainable number of the model. M is a positive integer whose value this book doesn't need. Training means changing θ again and again so that the model's answers on training pairs get better. Nanda's model also has a weight matrix called W_O (the attention output matrix); yours doesn't. That's a known gap.

N6 f**, the train fraction.** Scalar, $f = 0.3$. It's hard-coded in `get_dataloaders` as `train_size = int(0.3 * len(...))`; the config entry `train_fraction` doesn't change it. Role: the share of pairs used for training.

N7 D_{train}, D_{test}, N_{train}, N_{test}**.** D_{train} is the training set: a set of $N_{\text{train}} = 3{,}830$ pairs that are used to change the weights. D_{test} is the test set: the other $N_{\text{test}} = 8{,}939$ pairs, never used to change the weights. N_{train} and N_{test} are integers (Section 2.2). Role: D_{test} measures whether the model has learned the rule.

N8 Seed i **and run.** A seed is an integer label $i \in \{0, 1, 2, 3, 4\}$ that fixes the random numbers of one run (the initial weights and the data split). A run is one full training of the model with one seed. Role: 5 seeds give 5 independent, repeatable runs.

Part C. The model and its training

N9 Epoch t **and full batch.** An epoch is one full pass over D_{train}. t is an integer epoch counter, $t \in \{0, 1, \ldots, 40{,}000\}$; $t = 0$ means before any training. Every run lasts 40,000 epochs. Your runs are full batch: all 3,830 training pairs are used together each time θ is changed, so θ changes once per epoch.

N10 θ_t**, the weights after epoch** t**.** Vector with M entries, $\theta_t = (\theta_{t,1}, \ldots, \theta_{t,M})$: the value of θ (**N5**) at the end of epoch t. θ_0 is the initial weights. Role: everything the model computes at epoch t comes from θ_t.

N11 z**, the logits.** Vector with $p = 113$ entries $z_0, \ldots, z_{112}$, computed by the model for one pair at θ_t. Entry z_k is the score for answer $k \in \{0, \ldots, 112\}$; a larger z_k means the model favours k more.

N12 $\hat{k}$**, the predicted answer.** Integer in $\{0, \ldots, 112\}$: the k with the largest z_k, at θ_t. The model is correct on a pair when $\hat{k} = c$.

N13 q_k**, the softmax probability.** Scalar in $(0, 1)$, computed from z at θ_t: $q_k = e^{z_k} / \sum_{j=0}^{112} e^{z_j}$, where j is a summation index over the 113 answers. The 113 values q_k add up to 1. q_c is the probability the model gives the correct answer. (The letter q is used so it doesn't clash with the modulus p or the event epoch P.)

N14 ℓ**, the loss of one pair (cross-entropy).** Scalar, $\ell = -\ln q_c$, at θ_t. Sign: $\ell \geq 0$; $\ell = 0$ when $q_c = 1$, and ℓ grows as q_c falls. Role: punishes a low probability on the correct answer.

N15 $L(\theta_t)$**, the training loss.** Scalar: the average of ℓ over the 3,830 pairs of D_{train}, at θ_t. Role: the number training tries to make small.

N16 $g(\theta_t)$**, the gradient.** Vector with M entries; entry m is $\partial L / \partial \theta_m$, the partial derivative (how fast L changes when only θ_m changes), evaluated at θ_t. Direction: $g(\theta_t)$ points uphill (the direction in which L grows fastest), so a step along $-g(\theta_t)$ goes downhill.

N17 η**, the learning rate.** Scalar, $\eta > 0$, the same at every epoch. Role: the step size of each update. Its value is in the config; this book doesn't need it.

N18 AdamW, β_1**,** β_2**,** u_t**.** AdamW is the optimiser: the method that turns gradients into weight updates [4]. It keeps running averages of the gradient and of the squared gradient; $\beta_1 = 0.9$ and $\beta_2 = 0.98$ are scalars that set how much of the past those averages keep. u_t is the AdamW step: a vector with M entries, computed at epoch t from $g(\theta_0), \ldots, g(\theta_t)$ and β_1, β_2. Direction: like g, it points roughly uphill, so $-\eta u_t$ is a step downhill.

N19 λ**, the weight decay.** Scalar, $\lambda = 1.0$, the same at every epoch. Sign: $\lambda \geq 0$. Role: shrinks every weight a little at each update.

N20 The weight update. $\theta_{t+1} = \theta_t - \eta\, u_t - \eta\lambda\, \theta_t$. The middle term is the AdamW step downhill. The last term is the decoupled weight decay: decoupled means it's applied directly to θ_t, separately from u_t. It pulls every weight towards 0 (Section 2.4). A confound is a second cause that can produce the same effect as the one you're testing; weight decay is one for any weight-based quantity.

N21 $\|\theta_t\|_2$ **and** S_t**.** $\|\theta_t\|_2 = \sqrt{\sum_{m=1}^{M} \theta_{t,m}^2}$ is the L2 norm (overall size) of the weights: a scalar ≥ 0 at epoch t. $S_t = \|\theta_t\|_2^2 = \sum_{m=1}^{M} \theta_{t,m}^2$ is its square, also a scalar ≥ 0. Nanda writes S_t as $\sum w^2$.

N22 Checkpoint. A saved copy of θ_t at one epoch t. Checkpoints are saved every 50 epochs and evaluated every 100 epochs; evaluating a checkpoint means computing from it the quantities that need saved weights. Role: some quantities are computed afterwards from checkpoints.

Part D. The outcome

N23 $\text{acc}_{\text{train}}(t)$**,** $\text{acc}_{\text{test}}(t)$**.** Scalars in $[0, 1]$: the fraction of pairs in D_{train} (or D_{test}) on which $\hat{k} = c$, using θ_t. Direction: higher is better. Role: acc_{test} measures how well the model does on pairs it was never trained on.

N24 Memorisation and generalisation. Memorisation: getting the training pairs right by storing them, so $\text{acc}_{\text{train}}(t)$ is high while $\text{acc}_{\text{test}}(t)$ stays low. Generalisation: getting new pairs right because the rule $(a + b) \bmod p$ has been learned, so $\text{acc}_{\text{test}}(t)$ is high too.

N25 Grokking. In one run, $\text{acc}_{\text{train}}(t)$ reaches about 1 early, and much later $\text{acc}_{\text{test}}(t)$ suddenly jumps to high values. It is a switch from memorisation to generalisation.

N26 Circuit and Nanda's three phases. A circuit is a set of weights in θ_t that together compute the general rule $(a+b)$ mod p. Nanda's three phases [2]: memorisation; circuit formation (the circuit is built while acc_{test} stays low); cleanup (the weights used for memorising are removed, and acc_{test} jumps).

N27 Grok threshold and grok epoch G_i. The grok threshold is the scalar 0.9 (the first entry of `grok_thresholds` in the config). G_i is an integer epoch: $G_i = \min\{t : \text{acc}_{\text{test}}(t) > 0.9\}$ in the run of seed i. Values: $(G_0, G_1, G_2, G_3, G_4) = (12987, 6921, 9510, 11019, 23747)$. Role: the epoch the benchmark tries to warn about in advance. "Grok" on its own ("before grok") means the grok epoch G_i of that run.

Part E. Predictors and events

N28 Signal y_t. A scalar computed at epoch t during training, from θ_t or from the model's outputs. Example: S_t or $\|\theta_t\|_2$. y_{final} is its value at the last epoch, $t = 40{,}000$. y_{peak} is its largest value over all epochs of the run.

N29 Moving average, window w, causal, centred. A moving average replaces each point of a signal curve by the average of w neighbouring points; w is a positive integer, the window. A causal moving average at epoch t uses only points at or before t. A centred one uses points on both sides of t, so it uses later points too. `uniform_filter1d` (the smoothing function in your code) is centred. An ex-ante trigger is a rule that uses only the past.

N30 Rule, fire, event epoch P_i, miss, `None`. A rule is a fixed procedure that reads a signal curve and names one epoch. If it names an epoch in the run of seed i, the rule fires, and that epoch is the event epoch P_i: a scalar (an epoch; it can be non-integer). If it never names an epoch, that's a miss, and the code stores `None` instead of a number.

N31 Predictor. A (signal, rule) pair. If the rule changes, it's a different predictor. You have 8 predictors, in this order: L2 Norm, Dropout, Spectral, AGE, HTSR Alpha, Weight-PCA, Higher-MI and Commutator Defect. This book uses only L2 Norm rules and one Dropout rule as examples; they're defined in the next entries. The other predictors are named only.

N32 Log-epoch grid. For the L2 Norm predictor, the moving averages are taken on a grid of 40,000 points spaced evenly in ln(epoch); 43.5% of them lie before epoch 100. Grid values between recorded epochs are interpolated (filled in from neighbouring recorded values). That's why L2 events have non-integer epochs such as 113.4.

N33 The L2 Norm rules. L2 Norm's signal is $\|\theta_t\|_2$, recorded every epoch. *Crossover rule*: two centred moving averages of the signal on the log-epoch grid, a fast one ($w = 50$) and a slow one ($w = 200$); it fires at the first epoch after the first 100 epochs where the fast average crosses from below to above the slow one. Its events are $P = (113.4, 113.2, 120.1, 130.6, 136.8)$ for seeds 0 to 4. *MA-of-MA zero crossing*: a moving average applied to a moving average ($w = 20$), firing where that curve crosses zero, again after the first 100 epochs. Its events are $P =$

$(1802, 1767, 1814, 1811, 1801)$. The exact curve that crosses zero is in the code file `l2_norm.py`; you only need these P values here. *Peak event*: the epoch where S_t is largest; it lies between epochs 1038 and 1061 in every seed. *Halfway reading*: the first epoch after the peak where S_t has fallen to $y_{\text{target}} = y_{\text{peak}} - 0.5(y_{\text{peak}} - y_{\text{final}})$, with $y_t = S_t$.

N34 Dropout 10% rule. A Dropout rule that fires when the Dropout signal has covered 10% of its rise, where the rise is measured up to the run's final value y_{final}.

N35 `skip_epochs`. Integer config setting: the number of first epochs a rule ignores. For L2 it's 100.

N36 Firing set F and n. F is the set of seeds whose rule fired: $F \subseteq \{0, 1, 2, 3, 4\}$. $n = \#F$ is an integer from 0 to 5. Seeds with `None` are dropped before anything is computed. If $n < 4$, the predictor is rejected automatically, whatever its other numbers are.

N37 Lead and the 100-epoch margin. Seed $i \in F$ leads if $P_i < G_i - 100$: the event comes more than 100 epochs before grok. 100 is the margin, an integer number of epochs.

N38 lead_frac**, the lead fraction.** Scalar in $[0, 1]$: lead_frac $= \#\{i \in F : P_i < G_i - 100\}/n$, the share of firing seeds that lead.

N39 r_i and $\bar{r}$, relative lead. $r_i = (G_i - P_i)/G_i$ is a scalar for seed $i \in F$. Sign: $r_i > 0$ when the event is before grok, $r_i < 0$ when after; r_i near 1 means the event is very early compared with grok. $\bar{r} = \frac{1}{n}\sum_{i \in F} r_i$ is the mean relative lead, a scalar.

N40 Rank $R(X_i)$ and ties. For a list of n numbers X_i, $R(X_i)$ is the position of X_i when the list is sorted from smallest to largest; the smallest has rank 1. A tie is two or more equal values; tied values all get the average of the positions they occupy.

N41 Mean $\bar{X}$ and Pearson correlation $C(X, Y)$. For two lists X_i, Y_i of scalars over the same n seeds, $\bar{X} = \frac{1}{n}\sum X_i$ (a scalar, the mean), and $C(X, Y) = \dfrac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum (X_i - \bar{X})^2 \sum (Y_i - \bar{Y})^2}}$, a scalar in $[-1, 1]$. It measures how close the points are to a straight line; $+1$ means a rising line. It is undefined if either list is constant (the denominator is 0); code then typically returns NaN, "not a number".

N42 Spearman's ρ. Scalar in $[-1, 1]$: $\rho = C\big(R(P), R(G)\big)$, the Pearson correlation of the ranks, over the n seeds in F. $+1$ means later grok always comes with a later event. A fixed clock is an event at about the same epoch in every run, whatever G_i is; it gives ρ near 0, or undefined. With no ties it equals $\rho = 1 - \dfrac{6\sum_{i \in F} d_i^2}{n(n^2 - 1)}$, where $d_i = R(G_i) - R(P_i)$ is an integer, the rank difference of seed i.

N43 Adjacent swap. Two seeds whose ranks are next to each other (e.g. 2 and 3) appear in the opposite order in P and in G.

N44 $n!$, chance and p-value. $n! = n \times (n - 1) \times \cdots \times 1$ is the number of orderings of n items. "By chance" means all $n!$ orderings of the events are equally likely. The p-value of a ρ is the probability, by chance, of a ρ at least that large. A ρ is called significant when its p-value is small.

Part F. Judging predictors, and research honesty

N45 PASS rule, FAIL, verdict. A predictor gets the verdict PASS only if all three hold: lead_frac ≥ 0.8, $\rho \geq 0.9$ and $0.05 \leq \bar{r} \leq 0.9$. Otherwise its verdict is FAIL. A predictor rejected automatically for $n < 4$ (**N36**) also gets FAIL. The function `evaluate_frozen_protocol` in `analyze_nanda_unified.py` (**A9**) applies this rule; scoring a predictor means running it. In the code the ρ cutoff is written as 0.8999999999999998 so that a ρ of 0.9 isn't lost to floating-point rounding.

N46 Frozen protocol. The fixed set of rules used to score every predictor: `nanda_unified.yaml`, $p = 113$, 5 seeds, grok threshold 0.9, and the PASS thresholds (**N45**). It was written on 29 September 2026 and isn't changed while predictors are scored.

N47 Closed-negative. A predictor whose study is finished with a FAIL verdict. The first seven of your eight predictors are closed-negative. "L2 was closed" means L2 was closed-negative, on 26 September 2026.

N48 Pre-registration, criterion validation, synthetic predictor, calibration, post-hoc. Pre-registration: writing down the test before running it. A synthetic predictor is a made-up list of event epochs built from known G_i, used to test the test. Criterion validation: checking that the PASS rule passes a good synthetic predictor and fails a bad one. Calibration: setting a threshold of the test using data. Post-hoc: decided after the results were seen.

N49 Look-ahead (hindsight) bias. Using information from later epochs (e.g. y_{final}) to decide when an earlier event happened. A rule free of it could be run live during training.

N50 Researcher degrees of freedom. Choices of windows, thresholds or `skip_epochs` made after looking at the results.

N51 Sensitivity analysis and near-miss. Sensitivity analysis: changing one setting a little and checking whether the verdict changes. A near-miss is a FAIL that misses a threshold by a small amount.

N52 Baseline (control) and noise floor. A control run is one where grokking doesn't happen, or a setting is switched off (e.g. $\lambda = 0$). The noise floor is how much a signal wobbles for no reason.

N53 Negative result and reproducibility. A negative result is a clear "this doesn't work". Reproducibility means someone else can rerun your work and get the same numbers.

N54 Summary table. The L2 results table of 26 September 2026. Its "before grok" column counts a seed as early if $G_i - P_i > 0$, not if $P_i < G_i - 100$.

Module 1

The grokking phenomenon

1.1 Grokking

Definition: Grokking (**N25**) is what happens when, in one run, training accuracy $\text{acc}_{\text{train}}(t)$ reaches about 1 early, and much later test accuracy $\text{acc}_{\text{test}}(t)$ suddenly jumps to high values.

Why it matters: It's the event your whole benchmark tries to predict. Every predictor is judged by whether it signals before the jump, measured by the grok epoch G_i (**N27**).

Worked example: In your runs, $\text{acc}_{\text{train}}(t)$ reaches 1 within a few hundred epochs, but $\text{acc}_{\text{test}}(t)$ only passes 0.9 at epochs 6921 to 23747, depending on the seed. There's no calculation here.

Self-check: (1) Why is grokking surprising? (The model already fits D_{train}, yet acc_{test} keeps improving thousands of epochs later.) (2) Which two curves do you need to see it? ($\text{acc}_{\text{train}}(t)$ and $\text{acc}_{\text{test}}(t)$.)

Likely viva question: "Define grokking precisely. How is it different from ordinary slow learning?" (In slow learning $\text{acc}_{\text{train}}$ and acc_{test} rise together. In grokking there's a long gap with $\text{acc}_{\text{train}} \approx 1$ and acc_{test} low, then acc_{test} jumps past 0.9 at G_i.)

1.2 Memorisation vs generalisation

Definition: Memorisation (**N24**) is getting the training pairs right by storing them: $\text{acc}_{\text{train}}$ is high while acc_{test} stays low. Generalisation is getting new pairs right because the rule $(a + b) \bmod p$ has been learned: acc_{test} is high too.

Why it matters: Grokking is the switch from memorisation to generalisation. A predictor that reacts to early memorisation instead of to that switch is warning about the wrong thing. The L2 crossover rule (**N33**) is an example, and that's an attack point (**A11**).

Worked example: The L2 crossover fires at epochs 113.4 to 136.8, while the weight norm is still rising during memorisation. The derivation shows how much earlier than grok that is, for seed 0.

Derivation (H.K. Dass layout): how much earlier the L2 crossover fires than grok, seed 0

Given:

1. $G_0 = 12987$ **(N27)**.
2. $P_0 = 113.4$, the crossover event of seed 0 (**N33**).

Let:

1. G_0: the grok epoch of seed 0. Scalar (an integer epoch), read from acc_{test} of run 0. Positive. Role: the target.
2. P_0: the crossover event epoch of seed 0. Scalar, read from the L2 signal of run 0. Positive. Role: the warning being judged.
3. κ_0: the ratio G_0/P_0. Scalar, positive. Role: how many times later grok comes than the event.

To find: κ_0.

Solution:

$$\begin{aligned} \kappa_0 &= \frac{G_0}{P_0} && \because \text{ definition of } \kappa_0 \text{ (Let 3)} \\ &= \frac{12987}{113.4} && \because \text{ substitution of Given 1 and Given 2} \\ &\approx 114.5 && \because \text{ division, digits after the 1st d.p. dropped} \end{aligned}$$

$\therefore$ In seed 0, grok comes about 114.5 times later than the crossover event. The same division for seeds 1 to 4 gives about 61.1, 79.2, 84.4 and 173.6.

$$\boxed{\kappa_0 = 114.5 \text{ (1 d.p.)}}$$

Self-check: At epoch 1,000, is a typical run memorising or generalising? How do you know? (Memorising: $\text{acc}_{\text{test}}(1000)$ is still below 0.9 in every seed, since the smallest G_i is 6921, and S_t is still near its peak at epochs 1038 to 1061.)

Likely viva question: "Is your signal detecting the switch to generalisation, or just the end of memorisation?"

1.3 Circuit formation and cleanup

Definition: Nanda et al. [2] describe three phases (**N26**): memorisation; circuit formation, where a circuit that computes $(a+b)$ mod p is built while acc_{test} stays low; and cleanup, where the weights used for memorising are removed and acc_{test} jumps.

Why it matters: It explains why a hidden signal could exist before G_i: the circuit is being built before the jump. That's the premise of the benchmark.

Worked example: Nanda report that $S_t = \sum w^2$ (**N21**) falls smoothly during circuit formation and sharply during cleanup, at the same time as grokking. In your runs S_t rises during memorisation and peaks at epochs 1038 to 1061 in every seed. There's no calculation here.

Self-check: Which phase would a good early-warning signal need to detect? (Circuit formation, because it comes before cleanup and the jump.)

Likely viva question: "Nanda says the transition can't be predicted ahead of time. Why did you expect it could?"

Module 2

Your training setup

2.1 Modular addition task

Definition: The model learns $c = (a + b) \bmod p$ **(N3)** for all inputs $a, b \in \{0, \ldots, p-1\}$, with $p = 113$ **(N1)**.

Why it matters: It's the standard grokking task of Power et al. [1] and Nanda et al. [2], so your results can be compared with theirs.

Worked example: The derivation counts the pairs and works out one answer.

Derivation (H.K. Dass layout): number of pairs and one answer

Given:

1. $p = 113$ (**N1**, `nanda_unified.yaml`).
2. Each of a and b takes p values, 0 to $p - 1$ **(N2)**.
3. Example inputs $a = 100$, $b = 50$.

Let:

1. p: the modulus. Integer, the same in every seed and epoch. Positive. Role: size of the task.
2. N: the number of all pairs. Integer, fixed before training. Positive. Role: the pool the split is cut from.
3. a, b: the inputs of one pair. Integers in $\{0, \ldots, 112\}$. Role: what the model reads.
4. c: the correct answer of that pair. Integer in $\{0, \ldots, 112\}$. Role: its label.

To find: N and c for $a = 100$, $b = 50$.

Solution:

$$
\begin{aligned}
N &= p \times p && \because\ p \text{ choices of } a \text{ times } p \text{ choices of } b \text{ (Given 2)} \\
&= 113 \times 113 && \because\ \text{substitution of Given 1} \\
&= 12769 && \because\ \text{multiplication} \\
c &= (a + b) \bmod p && \because\ \text{definition of } c \text{ (\textbf{N3})} \\
&= (100 + 50) \bmod 113 && \because\ \text{substitution of Given 1 and Given 3} \\
&= 150 \bmod 113 && \because\ \text{addition} \\
&= 150 - 1 \times 113 && \because\ 113 \text{ fits once into } 150 \text{ (\textbf{A4})} \\
&= 37 && \because\ \text{subtraction}
\end{aligned}
$$

$\therefore$ There are 12,769 pairs, and the pair $(100, 50)$ has the answer 37.

$$\boxed{N = 12769, \quad c = 37}$$

Self-check: How many possible answers does the model choose between? ($p = 113$: the integers 0 to 112, one logit each.)

Likely viva question: "Why $p = 113$ and not another prime?" (It matches Nanda et al. 2023 [2].)

2.2 Train/test split (30%)

Definition: A fraction $f = 0.3$ (**N6**) of the N pairs forms the training set D_{train}; the other 70% form the test set D_{test} (**N7**), which is never used to change the weights.

Why it matters: G_i is measured on D_{test}, so the split must be fixed per seed and no test pair may leak into training. In your code the 30% is hard-coded in `get_dataloaders`; the config's `train_fraction` doesn't change it. That's an audit finding.

Worked example: The derivation computes the two set sizes.

Derivation (H.K. Dass layout): sizes of the training and test sets

Given:

1. $N = 12769$ (Section 2.1).
2. $f = 0.3$, hard-coded as `train_size = int(0.3 * len(...))` (**N6**).
3. `int()` of a positive number equals its floor (**A3**).

Let:

1. N: the number of all pairs. Integer, fixed before training. Role: the pool being split.
2. f: the train fraction. Scalar in $(0, 1)$. Role: share of pairs used for training.
3. N_{train}: the size of D_{train}. Integer, fixed per seed before training. Role: pairs used to change the weights.
4. N_{test}: the size of D_{test}. Integer, fixed per seed before training. Role: pairs that measure generalisation.

To find: N_{train} and N_{test}.

Solution:

$$
\begin{aligned}
N_{\text{train}} &= \lfloor f \times N \rfloor && \because \texttt{int(0.3 * len(...))} \text{ and Given 3} \\
&= \lfloor 0.3 \times 12769 \rfloor && \because \text{substitution of Given 1 and Given 2} \\
&= \lfloor 3830.7 \rfloor && \because \text{multiplication} \\
&= 3830 && \because \text{floor drops the 0.7 (\textbf{A3})} \\
N_{\text{test}} &= N - N_{\text{train}} && \because \text{every pair not in } D_{\text{train}} \text{ is in } D_{\text{test}} \\
&= 12769 - 3830 && \because \text{substitution of Given 1 and the line above} \\
&= 8939 && \because \text{subtraction}
\end{aligned}
$$

$\therefore$ Each run trains on 3,830 pairs and tests on 8,939 pairs.

$$\boxed{N_{\text{train}} = 3830, \quad N_{\text{test}} = 8939}$$

Self-check: What goes wrong if a training pair appears in D_{test}? (The model can get it right by memorising, so acc_{test} overstates generalisation and G_i comes too early.)

Likely viva question: "How do you guarantee the test set is truly unseen, and that it's the same split when you reload a checkpoint?" (The seed fixes the split; rebuilding it with the same seed gives the same D_{test}.)

2.3 Transformer, logits and cross-entropy loss

Definition: The model (**N5**) is a one-layer, four-head transformer. For one pair it reads (a, b) and the weights θ_t (**N10**) and returns the logits z (**N11**), a vector with 113 entries, one score per possible answer. The softmax turns them into probabilities $q_k = e^{z_k} / \sum_{j=0}^{112} e^{z_j}$ (**N13**). The loss of the pair is the cross-entropy $\ell = -\ln q_c$ (**N14**), and the training loss $L(\theta_t)$ is the average of ℓ over D_{train} (**N15**).

Why it matters: Every predictor is computed from this shared model's weights θ_t or its outputs.

Worked example: Two derivations: the loss at two probabilities, and why adding one constant to all logits leaves the loss unchanged.

Derivation (H.K. Dass layout): the loss at $q_c = 0.25$ and at $q_c = 0.99$

Given:

1. $\ell = -\ln q_c$ (**N14**).
2. Two example probabilities of the correct answer: 0.25 and 0.99.

Let:

1. q_c: the probability the model gives the correct answer c of one pair, at θ_t. Scalar in $(0, 1)$. Role: what the loss judges.
2. $\ell_{0.25}$: the loss of that pair when $q_c = 0.25$. Scalar ≥ 0. Role: loss of a poor answer.
3. $\ell_{0.99}$: the loss of that pair when $q_c = 0.99$. Scalar ≥ 0. Role: loss of a confident correct answer.

To find: $\ell_{0.25}$ and $\ell_{0.99}$.

Solution:

$$\begin{aligned}
\ell_{0.25} &= -\ln 0.25 && \because \text{ Given 1 with } q_c = 0.25 \\
&= -(-1.3863\ldots) && \because \ln 0.25 = -1.3863\ldots \\
&= 1.3863\ldots && \because -(-x) = x \\
\ell_{0.99} &= -\ln 0.99 && \because \text{ Given 1 with } q_c = 0.99 \\
&= -(-0.01005\ldots) && \because \ln 0.99 = -0.01005\ldots
\end{aligned}$$

$$= 0.01005\ldots \qquad \because\ -(-x) = x$$

$\therefore$ A probability of 0.25 on the correct answer costs about 1.39; a probability of 0.99 costs about 0.01. (Here "..." means the digits go on; the boxed values are rounded.)

$$\boxed{\ell_{0.25} = 1.39 \text{ (2 d.p.)}, \quad \ell_{0.99} = 0.01 \text{ (2 d.p.)}}$$

Derivation (H.K. Dass layout): adding the same constant to all logits doesn't change the loss

Given:

1. $q_k = e^{z_k} / \sum_{j=0}^{112} e^{z_j}$ (**N13**).
2. $\ell = -\ln q_c$ (**N14**).
3. $e^{x+y} = e^x e^y$ for scalars x, y (rule of exponents).

Let:

1. z: the logits of one pair at θ_t. Vector with 113 entries $z_0, \ldots, z_{112}$. Role: the model's scores.
2. c: the correct answer of that pair. Integer in $\{0, \ldots, 112\}$. Role: picks which probability the loss uses.
3. s: a constant. Scalar, any sign, the same for all 113 entries. Role: the shift being tested.
4. z': the shifted logits, $z'_k = z_k + s$ for every k. Vector with 113 entries. Role: the changed input.
5. q'_c: the softmax probability of c computed from z'. Scalar in $(0, 1)$.
6. ℓ': the loss computed from z', $\ell' = -\ln q'_c$. Scalar ≥ 0.

To find: Show that $\ell' = \ell$.

Solution:

$$\begin{aligned}
q'_c &= \frac{e^{z'_c}}{\sum_j e^{z'_j}} && \because \text{ Given 1 applied to } z' \\
&= \frac{e^{z_c+s}}{\sum_j e^{z_j+s}} && \because \text{ substitution } z'_k = z_k + s \text{ (Let 4)} \\
&= \frac{e^s e^{z_c}}{\sum_j e^{z_j+s}} && \because \text{ Given 3 on the numerator} \\
&= \frac{e^s e^{z_c}}{\sum_j e^s e^{z_j}} && \because \text{ Given 3 on each term of the denominator} \\
&= \frac{e^s e^{z_c}}{e^s \sum_j e^{z_j}} && \because\ e^s \text{ is the same scalar in every term, so it comes out of the sum} \\
&= \frac{e^{z_c}}{\sum_j e^{z_j}} && \because\ e^s/e^s = 1, \text{ since } e^s > 0 \\
&= q_c && \because \text{ Given 1} \\
\ell' &= -\ln q'_c && \because \text{ definition of } \ell' \text{ (Let 6)}
\end{aligned}$$

$$= -\ln q_c \qquad \because\ q'_c = q_c\text{, shown above}$$
$$= \ell \qquad \because\ \text{Given 2}$$

$\therefore$ Adding the same number to all 113 logits leaves every probability, and so the loss, unchanged.

$$\boxed{\ell' = \ell}$$

Self-check: Why does adding the same constant to all 113 logits not change the loss? (The derivation above: e^s cancels.)

Likely viva question: "Is your model exactly Nanda's?" (No. It lacks the attention output matrix W_O (**N5**). That's a known gap.)

2.4 AdamW and weight decay

Definition: AdamW (**N18**) is the optimiser that turns gradients into weight updates. Weight decay λ (**N19**) shrinks every weight a little at each update. The full update is the equation $\theta_{t+1} = \theta_t - \eta\, u_t - \eta\lambda\, \theta_t$ (**N20**).

Why it matters: You use $\lambda = 1.0$ with $\beta_1 = 0.9$, $\beta_2 = 0.98$, as in Nanda. Weight decay is believed to drive grokking. It can also make a weight-based signal such as $\|\theta_t\|_2$ fall on its own. So a falling norm could be caused by the optimiser rather than be a sign of grokking. That's a confound: a second cause that can produce the same effect.

Worked example: The derivation rewrites the update to show the decay as a shrink factor.

Derivation (H.K. Dass layout): the weight decay as a shrink factor

Given:

1. $\theta_{t+1} = \theta_t - \eta\, u_t - \eta\lambda\, \theta_t$ (**N20**).
2. $\lambda = 1.0$ (**N19**, `nanda_unified.yaml`).

Let:

1. θ_t: the weights after epoch t. Vector with M entries. Role: what's being updated.
2. θ_{t+1}: the weights after epoch $t+1$. Vector with M entries.
3. u_t: the AdamW step at epoch t. Vector with M entries, roughly uphill, so $-\eta u_t$ is downhill.
4. η: the learning rate. Scalar > 0.
5. λ: the weight decay. Scalar ≥ 0.

To find: θ_{t+1} written as a scaled copy of θ_t plus the AdamW step, for $\lambda = 1.0$.

Solution:

$$\theta_{t+1} = \theta_t - \eta\, u_t - \eta\lambda\, \theta_t \qquad \because\ \text{Given 1}$$
$$= \theta_t - \eta\lambda\, \theta_t - \eta\, u_t \qquad \because\ \text{reordering the two subtracted terms}$$

$$
\begin{aligned}
&= (1-\eta\lambda)\,\theta_t - \eta\,u_t && \because \text{ factoring the scalar out of } \theta_t - \eta\lambda\theta_t \\
&= (1-\eta\times 1.0)\,\theta_t - \eta\,u_t && \because \text{ substitution of Given 2} \\
&= (1-\eta)\,\theta_t - \eta\,u_t && \because\ \eta\times 1.0 = \eta
\end{aligned}
$$

$\therefore$ At every update the weights are first scaled by $(1-\eta)$, which is below 1, and then moved by the AdamW step. The scaling pulls $\|\theta_t\|_2$ down whatever the data does.

$$\boxed{\theta_{t+1} = (1-\eta)\,\theta_t - \eta\,u_t}$$

Self-check: Why could a falling weight norm be "caused by the optimiser" rather than "a sign of grokking"? (The factor $(1-\eta)$ shrinks θ_t at every update, in every run.)

Likely viva question: "Did you vary λ? If not, how can you separate your signal from the effect of weight decay?"

2.5 Epochs and checkpoints

Definition: An epoch (**N9**) is one full pass over D_{train}. Your runs are full batch, so one epoch is one weight update. A checkpoint (**N22**) is a saved copy of θ_t at one epoch.

Why it matters: All runs last 40,000 epochs. G_i and every event P_i are reported in epochs, and several predictors are computed afterwards from checkpoints. Checkpoints are saved every 50 epochs and evaluated every 100, so a predictor computed from checkpoint evaluations can only place its event to within 100 epochs.

Worked example: The derivation finds the two checkpoint evaluations around G_0, assuming evaluations fall on multiples of 100.

Derivation (H.K. Dass layout): checkpoint evaluations around G_0

Given:

1. $G_0 = 12987$ **(N27)**.
2. Checkpoint evaluations every 100 epochs, taken here to be at multiples of 100 (**N22**).

Let:

1. G_0: the grok epoch of seed 0. Integer epoch.
2. t_-: the last evaluated epoch at or before G_0. Integer, a multiple of 100.
3. t_+: the first evaluated epoch after G_0. Integer, a multiple of 100.

To find: t_- and t_+.

Solution:

$$
\begin{aligned}
t_- &= 100 \times \left\lfloor \frac{G_0}{100} \right\rfloor && \because \text{ largest multiple of 100 that's } \le G_0 \textbf{ (A3)} \\
&= 100 \times \lfloor 129.87 \rfloor && \because \text{ substitution of Given 1, and } 12987/100 = 129.87
\end{aligned}
$$

$$
\begin{aligned}
&= 100 \times 129 && \because \text{ floor} \\
&= 12900 && \because \text{ multiplication} \\
t_+ &= t_- + 100 && \because \text{ evaluations are 100 epochs apart (Given 2)} \\
&= 13000 && \because \text{ addition}
\end{aligned}
$$

$\therefore$ From checkpoint evaluations alone you'd only know that seed 0 groks between epochs 12,900 and 13,000.

$$\boxed{t_- = 12900, \quad t_+ = 13000}$$

Your G_i values (e.g. 12987) aren't multiples of 50 or 100, so they come from an accuracy curve recorded more often than the checkpoints. Check which curve `analyze_nanda_unified.py` reads.

Self-check: Why must the data split be rebuilt with the same seed when re-reading a checkpoint? (Otherwise D_{test} would contain pairs the checkpoint was trained on.)

Likely viva question: "What's your time resolution, and could it change any verdict?" (A margin of 100 epochs (**N37**) is the same size as the evaluation spacing.)

2.6 Seeds and why five

Definition: A seed i (**N8**) fixes the random numbers of one run (the initial weights and the data split), so each seed is one independent, repeatable run.

Why it matters: Your benchmark uses 5 seeds, $i \in \{0, 1, 2, 3, 4\}$. Different seeds grok at different epochs, and that spread is what lets you test whether a predictor tracks when grok happens, not just that it happens.

Worked example: The derivation computes the mean grok epoch.

Derivation (H.K. Dass layout): the mean grok epoch over the five seeds

Given:

1. $(G_0, G_1, G_2, G_3, G_4) = (12987, 6921, 9510, 11019, 23747)$ **(N27)**.
2. The mean of 5 numbers is their sum divided by 5 (**N41**).

Let:

1. G_i: the grok epoch of seed i. Integer epoch, positive.
2. $\bar{G}$: the mean grok epoch. Scalar epoch, positive. Role: a typical grok time.

To find: $\bar{G}$.

Solution:

$$
\begin{aligned}
\bar{G} &= \frac{G_0 + G_1 + G_2 + G_3 + G_4}{5} && \because \text{ Given 2} \\
&= \frac{12987 + 6921 + 9510 + 11019 + 23747}{5} && \because \text{ substitution of Given 1}
\end{aligned}
$$

$$= \frac{64184}{5} \qquad \because \text{ addition}$$

$$= 12836.8 \qquad \because \text{ division}$$

$\therefore$ The five runs grok at epoch 12,836.8 on average, ranging from 6,921 (seed 1) to 23,747 (seed 4).

$$\boxed{\bar{G} = 12836.8}$$

Self-check: Why is one seed not enough to judge a predictor? (One run gives one (P_i, G_i) pair, so you can't tell whether P moves with G.)

Likely viva question: "Five is very few. What can and can't you conclude from five seeds?" (See Module 5.)

Module 3

Measuring the outcome

3.1 Test accuracy

Definition: Test accuracy $\text{acc}_{\text{test}}(t)$ (**N23**) is the fraction of the 8,939 test pairs on which the predicted answer $\hat{k}$ (**N12**) equals the correct answer c, using θ_t.

Why it matters: It's the only thing that defines grok in your benchmark.

Worked example: The derivation finds how many test pairs must be right for acc_{test} to pass the grok threshold 0.9.

Derivation (H.K. Dass layout): correct test pairs needed for $\text{acc}_{\text{test}} > 0.9$

Given:

1. $N_{\text{test}} = 8939$ (Section 2.2).
2. $\text{acc}_{\text{test}}(t) = (\text{number of correct test pairs})/N_{\text{test}}$ (**N23**).
3. Grok needs $\text{acc}_{\text{test}}(t) > 0.9$ (**N27**).

Let:

1. K: the number of test pairs with $\hat{k} = c$ at θ_t. Integer from 0 to 8939. Role: the count behind acc_{test}.
2. $K_{\min}$: the smallest K with $K/N_{\text{test}} > 0.9$. Integer.

To find: $K_{\min}$.

Solution:

$\frac{K}{N_{\text{test}}} > 0.9$	$\because$ Given 2 put into Given 3
$K > 0.9 \times N_{\text{test}}$	$\because$ multiplying both sides by $N_{\text{test}} > 0$
$K > 0.9 \times 8939$	$\because$ substitution of Given 1
$K > 8045.1$	$\because$ multiplication
$K_{\min} = 8046$	$\because$ K is an integer; 8046 is the first integer above 8045.1

$\therefore$ At least 8,046 of the 8,939 test pairs must be right. For example, 8,100 right gives $8100/8939 = 0.906$ (3 d.p.), which passes.

$$\boxed{K_{\min} = 8046}$$

Self-check: Why not use test loss instead? (Loss also changes when the model just gets more or less confident on pairs it already gets right; accuracy only changes when answers change.)

Likely viva question: "Why accuracy and not loss to define grokking?"

3.2 The grok epoch G_i

Definition: The grok epoch of seed i is $G_i = \min\{t : \text{acc}_{\text{test}}(t) > 0.9\}$ **(N27)**: the first epoch at which test accuracy goes above 0.9.

Why it matters: It's the target every predictor is compared with. Recomputing it from the saved curves reproduces your saved values.

Worked example: $(G_0, G_1, G_2, G_3, G_4) = (12987, 6921, 9510, 11019, 23747)$. The derivation measures their spread.

Derivation (H.K. Dass layout): the spread of the grok epochs

Given:

1. $(G_0, \ldots, G_4) = (12987, 6921, 9510, 11019, 23747)$ **(N27)**.

Let:

1. $G_{\max}$: the largest G_i. Integer epoch.
2. $G_{\min}$: the smallest G_i. Integer epoch.
3. Δ: the range, $G_{\max} - G_{\min}$. Integer number of epochs, ≥ 0. Role: how far apart the grok times are.

To find: Δ.

Solution:

$$
\begin{aligned}
G_{\max} &= 23747 && \because \text{ seed 4 has the largest value in Given 1} \\
G_{\min} &= 6921 && \because \text{ seed 1 has the smallest value in Given 1} \\
\Delta &= G_{\max} - G_{\min} && \because \text{ definition of } \Delta \text{ (Let 3)} \\
&= 23747 - 6921 && \because \text{ substitution of the two lines above} \\
&= 16826 && \because \text{ subtraction}
\end{aligned}
$$

$\therefore$ The grok epochs spread over 16,826 epochs, which is what gives ρ (Module 5) something to track.

$$\boxed{\Delta = 16826}$$

Self-check: Would G_i change much with a 0.95 threshold? Why does it matter that the threshold is fixed? (It moves G_i later by however long acc_{test} takes to go from 0.9 to 0.95; fixing it stops it being picked to suit a predictor.)

Likely viva question: "Why 0.9? Is your result sensitive to that choice?"

Module 4

What a predictor is

4.1 Predictor and event epoch P_i

Definition: A predictor (**N31**) is a (signal, rule) pair: a signal y_t (**N28**) tracked during training, plus a rule (**N30**) that names one epoch P_i, the event epoch, as the warning in the run of seed i. If the rule never names an epoch, that's a miss, stored as `None`.

Why it matters: Each of your eight predictors is a (signal, rule) pair. If the rule changes, it's a different predictor, so every verdict belongs to one exact rule.

Worked example: The L2 crossover rule (**N33**) gives $P = (113.4, 113.2, 120.1, 130.6, 136.8)$ for seeds 0 to 4. They aren't whole epochs because L2's averages are taken on the log-epoch grid (**N32**). Rounded to whole epochs they're $(113, 113, 120, 131, 137)$. This book always uses the unrounded values; Section 5.2 shows why rounding matters. There's no calculation here.

Self-check: What should the code do if the rule never fires? (Store `None`; the seed is then dropped from F, **N36**.)

Likely viva question: "Which exact rule is the official one for this predictor?" (For L2 none was ever named: the crossover and the MA-of-MA crossing were both implemented.)

4.2 Lead and the 100-epoch margin

Definition: A firing seed i leads (**N37**) only if its event comes more than 100 epochs before grok: $P_i < G_i - 100$.

Why it matters: The margin stops an event that fires at grok, or just before it, from counting as a warning.

Worked example: The derivation checks seed 0 of the L2 crossover, and a made-up event at epoch 12950 for contrast.

Derivation (H.K. Dass layout): does seed 0 lead?

Given:

1. $G_0 = 12987$ (**N27**).
2. $P_0 = 113.4$ (**N33**).
3. Seed i leads if $P_i < G_i - 100$ (**N37**).

Let:

1. G_0: the grok epoch of seed 0. Integer epoch.
2. P_0: the crossover event epoch of seed 0. Scalar epoch.

3. T_0: the latest allowed event epoch for seed 0, $T_0 = G_0 - 100$. Scalar epoch. Role: the cutoff an event must be below.

To find: Whether $P_0 < T_0$.

Solution:

$$
\begin{aligned}
T_0 &= G_0 - 100 && \because \text{ definition of } T_0 \text{ (Let 3)} \\
&= 12987 - 100 && \because \text{ substitution of Given 1} \\
&= 12887 && \because \text{ subtraction} \\
P_0 &= 113.4 && \because \text{ Given 2} \\
113.4 &< 12887 && \because \text{ comparing the two numbers}
\end{aligned}
$$

$\therefore$ Seed 0 leads. An event at $P_0 = 12950$ wouldn't, because $12950 > 12887$.

$$\boxed{P_0 = 113.4 < T_0 = 12887 : \text{ seed 0 leads}}$$

Self-check: Does the summary table's "before grok" column use the same margin? (No. It counts $G_i - P_i > 0$ (**N54**). That's an audit finding: the halfway reading shows 5/5 there but 4/5 under the protocol, because seed 2 leads by only 13 epochs.)

Likely viva question: "Why 100 epochs?"

4.3 Lead fraction

Definition: The lead fraction (**N38**) is the share of firing seeds that lead: lead_frac $= \#\{i \in F : P_i < G_i - 100\}/n$, where F is the firing set and $n = \#F$ (**N36**).

Why it matters: PASS needs lead_frac ≥ 0.8. With $n = 5$ that's at least 4 of 5 seeds; with $n = 4$ it's 4 of 4.

Worked example: The derivation computes lead_frac for the L2 crossover, where all 5 seeds fire.

Derivation (H.K. Dass layout): lead_frac **of the L2 crossover**

Given:

1. $(G_0, \ldots, G_4) = (12987, 6921, 9510, 11019, 23747)$ (**N27**).
2. $(P_0, \ldots, P_4) = (113.4, 113.2, 120.1, 130.6, 136.8)$ (**N33**).
3. All 5 seeds fire, so $F = \{0, 1, 2, 3, 4\}$ (Given 2 has no `None`).
4. $(G_0 - 100, \ldots, G_4 - 100) = (12887, 6821, 9410, 10919, 23647)$ (subtracting 100 from each entry of Given 1).
5. lead_frac $= \#\{i \in F : P_i < G_i - 100\}/n$ (**N38**).

Let:

1. G_i, P_i: grok epoch and event epoch of seed i. Scalars.
2. n: the number of firing seeds. Integer.

3. $\mathcal{L}$: the set of seeds in F that lead. Set of seed labels. Role: the count on top of lead_frac.

To find: lead_frac.

Solution:

$$
\begin{aligned}
n &= \#F && \because \text{ definition of } n \textbf{ (N36)} \\
&= 5 && \because \text{ Given 3} \\
P_0 = 113.4 &< 12887 = G_0 - 100 && \because \text{ Given 2 and 4: seed 0 leads} \\
P_1 = 113.2 &< 6821 = G_1 - 100 && \because \text{ Given 2 and 4: seed 1 leads} \\
P_2 = 120.1 &< 9410 = G_2 - 100 && \because \text{ Given 2 and 4: seed 2 leads} \\
P_3 = 130.6 &< 10919 = G_3 - 100 && \because \text{ Given 2 and 4: seed 3 leads} \\
P_4 = 136.8 &< 23647 = G_4 - 100 && \because \text{ Given 2 and 4: seed 4 leads} \\
\#\mathcal{L} &= 5 && \because \text{ all five lines above hold} \\
\text{lead_frac} &= \frac{\#\mathcal{L}}{n} && \because \text{ Given 5} \\
&= \frac{5}{5} && \because \text{ substitution of the lines above} \\
&= 1.0 && \because \text{ division}
\end{aligned}
$$

$\therefore$ Every seed leads, so the L2 crossover meets the lead condition.

$$\boxed{\text{lead_frac} = 1.0}$$

Self-check: If one seed misses and one fires after grok, what is lead_frac? (The miss is dropped, so $n = 4$, and 3 of those 4 lead: lead_frac $= 3/4 = 0.75$, a fail.)

Likely viva question: "Is 4 out of 5 strong evidence?"

4.4 Relative lead and mean relative lead

Definition: The relative lead of seed i **(N39)** is $r_i = (G_i - P_i)/G_i$: how early the event is, as a fraction of the grok epoch. The mean relative lead is $\bar{r} = \frac{1}{n}\sum_{i \in F} r_i$.

Why it matters: PASS needs $0.05 \leq \bar{r} \leq 0.9$. Below 0.05 the event is too late to be useful. Above 0.9 it's so early that it's probably unrelated to grok.

Worked example: Two derivations: r_0 for the L2 crossover, then $\bar{r}$ over all five seeds.

Derivation (H.K. Dass layout): relative lead of seed 0, L2 crossover

Given:

1. $G_0 = 12987$ **(N27)**.
2. $P_0 = 113.4$ **(N33)**.
3. $r_i = (G_i - P_i)/G_i$ **(N39)**.

Let:

1. G_0: grok epoch of seed 0. Integer epoch, positive.
2. P_0: crossover event epoch of seed 0. Scalar epoch, positive.
3. r_0: relative lead of seed 0. Scalar; positive because $P_0 < G_0$.

To find: r_0.

Solution:

$$
\begin{aligned}
r_0 &= \frac{G_0 - P_0}{G_0} && \because \text{Given 3 with } i = 0 \\
&= \frac{12987 - 113.4}{12987} && \because \text{substitution of Given 1 and Given 2} \\
&= \frac{12873.6}{12987} && \because \text{subtraction} \\
&\approx 0.9913 && \because \text{division, digits after the 4th d.p. dropped}
\end{aligned}
$$

$\therefore$ Seed 0's event comes after only about 0.9% of the time to grok has passed.

$$\boxed{r_0 = 0.9913 \text{ (4 d.p.)}}$$

Derivation (H.K. Dass layout): mean relative lead of the L2 crossover

Given:

1. $(G_0, \ldots, G_4) = (12987, 6921, 9510, 11019, 23747)$ **(N27)**.
2. $(P_0, \ldots, P_4) = (113.4, 113.2, 120.1, 130.6, 136.8)$ **(N33)**.
3. $n = 5$ and $F = \{0, 1, 2, 3, 4\}$ (Section 4.3).
4. $\bar{r} = \frac{1}{n} \sum_{i \in F} r_i$ with $r_i = (G_i - P_i)/G_i$ **(N39)**.

Let:

1. r_i: relative lead of seed i. Scalar.
2. $\bar{r}$: mean relative lead. Scalar. Role: the third PASS condition.
3. S: the sum $\sum_{i=0}^{4} P_i/G_i$. Scalar, positive. Role: the part of $\bar{r}$ that depends on the events.

To find: $\bar{r}$.

Solution:

$$
\begin{aligned}
r_i &= \frac{G_i - P_i}{G_i} && \because \text{Given 4} \\
&= \frac{G_i}{G_i} - \frac{P_i}{G_i} && \because \text{splitting the fraction} \\
&= 1 - \frac{P_i}{G_i} && \because G_i/G_i = 1 \\
\bar{r} &= \frac{1}{5} \sum_{i=0}^{4} \left(1 - \frac{P_i}{G_i}\right) && \because \text{Given 4 with Given 3, and the line above} \\
&= \frac{1}{5} \left(5 - \sum_{i=0}^{4} \frac{P_i}{G_i}\right) && \because \text{the five 1s add up to 5}
\end{aligned}
$$

$$= 1 - \frac{1}{5}\sum_{i=0}^{4}\frac{P_i}{G_i} \qquad \because \text{multiplying out } \tfrac{1}{5}$$

$$= 1 - \frac{S}{5} \qquad \because \text{definition of } S \text{ (Let 3)}$$

$$S = \tfrac{113.4}{12987} + \tfrac{113.2}{6921} + \tfrac{120.1}{9510} + \tfrac{130.6}{11019} + \tfrac{136.8}{23747} \qquad \because \text{Let 3 with Given 1 and 2}$$

$$\approx 0.00873 + 0.01636 + 0.01263 + 0.01185 + 0.00576 \qquad \because \text{each fraction rounded to 5 d.p.; later digits dropped}$$

$$= 0.05533 \qquad \because \text{addition}$$

$$\bar{r} = 1 - \frac{0.05533}{5} \qquad \because \text{substitution of } S \text{ into the line } \bar{r} = 1 - S/5$$

$$= 1 - 0.011066 \qquad \because \text{division by 5}$$

$$= 0.988934 \qquad \because \text{subtraction}$$

$\therefore$ The mean relative lead is 0.989 to 3 d.p., which is above the upper limit 0.9, so the L2 crossover fails this condition.

$$\boxed{\bar{r} = 0.989 \text{ (3 d.p.)} > 0.9}$$

Self-check: What $\bar{r}$ does an event exactly halfway to grok give? ($P_i = 0.5\,G_i$ gives $r_i = 0.5$ in every seed, so $\bar{r} = 0.5$.)

Likely viva question: "Why have an upper limit at all?" (Without it, an event at epoch 1 in every run would score $\bar{r} \approx 1$ and look like the best possible warning.)

4.5 Misses and `None`

Definition: A miss (**N30**) is a seed where the rule never fires; the code stores `None`. Seeds with `None` are dropped before lead_frac, ρ and $\bar{r}$ are computed (**N36**). If fewer than 4 seeds fire ($n < 4$), the predictor fails automatically.

Why it matters: How misses are handled changes lead_frac. Because a miss is dropped rather than counted as "no lead", four leads and one miss give lead_frac $= 4/4$, not $4/5$. That's a possible attack point.

Worked example: The derivation works through one miss and four leads.

Derivation (H.K. Dass layout): one miss and four leads

Given:

1. One seed returns `None`; the other four fire and all four lead (example case).
2. Seeds with `None` are dropped; $n < 4$ is an automatic fail (**N36**).
3. lead_frac $= \#\{i \in F : P_i < G_i - 100\}/n$ (**N38**).

Let:

1. F: the firing set. Set of seed labels.
2. n: the number of firing seeds, $n = \#F$. Integer.

3. $\mathcal{L}$: the seeds in F that lead. Set of seed labels.

To find: Whether the predictor fails automatically, and its lead_frac.

Solution:

$$
\begin{aligned}
n &= 5 - 1 && \because \text{ one of the 5 seeds is dropped (Given 1, 2)} \\
&= 4 && \because \text{ subtraction} \\
n &\geq 4 && \because \text{ so the automatic fail of Given 2 doesn't apply} \\
\#\mathcal{L} &= 4 && \because \text{ all four firing seeds lead (Given 1)} \\
\text{lead_frac} &= \frac{\#\mathcal{L}}{n} && \because \text{ Given 3} \\
&= \frac{4}{4} && \because \text{ substitution} \\
&= 1.0 && \because \text{ division}
\end{aligned}
$$

$\therefore$ The predictor isn't failed automatically, and its lead fraction is 1.0, the same as if all five seeds had led.

$$\boxed{n = 4, \quad \text{lead_frac} = 1.0}$$

Self-check: Can you compute ρ with 4 seeds? (Yes: ρ is then computed on the 4 firing seeds, with $n = 4$ in **N42**.)

Likely viva question: "How do you treat a seed where the signal never appears?"

Module 5

Spearman's ρ and small samples

5.1 Spearman rank correlation ρ

Definition: Spearman's ρ (**N42**) is a number from -1 to 1 that says whether two lists go up together in order, using only their ranks (**N40**). With no ties, $\rho = 1 - 6\sum_{i\in F} d_i^2/\big(n(n^2-1)\big)$, where $d_i = R(G_i) - R(P_i)$ is the rank difference of seed i.

Why it matters: PASS needs $\rho \geq 0.9$: seeds that grok later must also have later events. That shows the event tracks this run's grok, not a fixed clock (an event at about the same epoch in every run, whatever G_i is).

Worked example: The derivation computes ρ for the L2 crossover. The five P_i are all different, so there are no ties.

Derivation (H.K. Dass layout): ρ of the L2 crossover

Given:

1. $(G_0, \ldots, G_4) = (12987, 6921, 9510, 11019, 23747)$ **(N27)**.
2. $(P_0, \ldots, P_4) = (113.4, 113.2, 120.1, 130.6, 136.8)$ **(N33)**.
3. $n = 5$ (Section 4.3).
4. Rank: position from smallest, starting at 1 (**N40**).
5. $\rho = 1 - 6\sum d_i^2/\big(n(n^2-1)\big)$ with $d_i = R(G_i) - R(P_i)$, when there are no ties (**N42**).

Let:

1. $R(G_i)$: rank of G_i among the five grok epochs. Integer from 1 to 5.
2. $R(P_i)$: rank of P_i among the five event epochs. Integer from 1 to 5.
3. d_i: rank difference of seed i. Integer; positive when seed i ranks higher in G than in P.
4. ρ: Spearman's rho of the L2 crossover. Scalar in $[-1, 1]$.

To find: ρ.

Solution:

$$\text{sorted } G: 6921 < 9510 < 11019 < 12987 < 23747 \quad \because \text{ sorting Given 1: seeds 1, 2, 3, 0, 4}$$

$$(R(G_0), \ldots, R(G_4)) = (4, 1, 2, 3, 5) \quad \because \text{ position of each seed in the line above}$$

$$\text{sorted } P: 113.2 < 113.4 < 120.1 < 130.6 < 136.8 \quad \because \text{ sorting Given 2: seeds 1, 0, 2, 3, 4}$$

$$(R(P_0), \ldots, R(P_4)) = (2, 1, 3, 4, 5) \quad \because \text{ position of each seed in the line above}$$

$$(d_0, \ldots, d_4) = (4-2,\ 1-1,\ 2-3,\ 3-4,\ 5-5) \quad \because\ d_i = R(G_i) - R(P_i) \text{ (Given 5)}$$

$$= (2, 0, -1, -1, 0) \quad \because \text{ subtraction}$$

$$\sum d_i^2 = 2^2 + 0^2 + (-1)^2 + (-1)^2 + 0^2 \quad \because \text{ squaring each } d_i$$

$$
\begin{aligned}
&= 4 + 0 + 1 + 1 + 0 && \because\ (-1)^2 = 1 \\
&= 6 && \because\ \text{addition} \\
\rho &= 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)} && \because\ \text{Given 5} \\
&= 1 - \frac{6 \times 6}{5(5^2 - 1)} && \because\ \text{substitution of } \textstyle\sum d_i^2 = 6 \text{ and Given 3} \\
&= 1 - \frac{6 \times 6}{5 \times 24} && \because\ 5^2 - 1 = 25 - 1 = 24 \\
&= 1 - \frac{36}{5 \times 24} && \because\ 6 \times 6 = 36 \\
&= 1 - \frac{36}{120} && \because\ 5 \times 24 = 120 \\
&= 1 - 0.3 && \because\ 36/120 = 0.3 \\
&= 0.7 && \because\ \text{subtraction}
\end{aligned}
$$

$\therefore$ The L2 crossover ranks the seeds with $\rho = 0.70$, below 0.9, so it fails the ρ condition.

$$\boxed{\rho = 0.70 < 0.9}$$

Self-check: What ρ does an event at a fixed epoch (the same in all seeds) get? (All P_i tie, so every $R(P_i)$ is 3, the $R(P)$ list is constant, and C in **N41** is undefined. Either way it can't reach 0.9, so it fails. Check whether your code returns NaN, "not a number", or 0 here.)

Likely viva question: "Why Spearman and not Pearson?"

5.2 Pearson vs Spearman

Definition: Pearson's correlation $C(X, Y)$ (**N41**) measures how close the points (X_i, Y_i) are to a straight line, using the raw values. Spearman's ρ is Pearson's correlation of the ranks, $\rho = C\big(R(P), R(G)\big)$, so it uses only order. When there are ties, this is how ρ is computed.

Why it matters: With 5 seeds, seed 4 ($G_4 = 23747$) is far from the rest and dominates Pearson's C. Spearman isn't pulled by one extreme value, because 23747 only counts as rank 5. Any event list that rises with G in exactly the same order has $\rho = 1$, even if the relationship is curved.

Worked example: The derivation shows how rounding the crossover events creates a tie and changes ρ.

Derivation (H.K. Dass layout): ρ of the L2 crossover with events rounded to whole epochs

Given:

1. Rounded events: $(P_0, \ldots, P_4) = (113, 113, 120, 131, 137)$ (Section 4.1).
2. $(R(G_0), \ldots, R(G_4)) = (4, 1, 2, 3, 5)$ (Section 5.1).
3. Tied values get the average of the positions they occupy (**N40**).
4. $\rho = C\big(R(P), R(G)\big)$, with C from **N41**; $n = 5$.

Let:

1. $X_i = R(P_i)$: rank of the rounded event of seed i. Scalar (can be a half).
2. $Y_i = R(G_i)$: rank of the grok epoch of seed i. Integer.
3. $\bar{X}$, $\bar{Y}$: the means of the X_i and of the Y_i. Scalars.
4. ρ: Spearman's rho with the tie. Scalar in $[-1, 1]$.

To find: ρ.

Solution:

$$
\begin{aligned}
(X_0, \ldots, X_4) &= (1.5, 1.5, 3, 4, 5) && \because \text{seeds 0 and 1 tie for positions 1 and 2 (Given 1, 3)} \\
\bar{X} &= \frac{1.5 + 1.5 + 3 + 4 + 5}{5} && \because \text{definition of the mean} \\
&= \frac{15}{5} && \because \text{addition} \\
&= 3 && \because \text{division} \\
\bar{Y} &= \frac{4 + 1 + 2 + 3 + 5}{5} && \because \text{Given 2 and the definition of the mean} \\
&= 3 && \because 15/5 = 3 \\
(X_i - \bar{X}) &= (-1.5, -1.5, 0, 1, 2) && \because \text{subtracting } \bar{X} = 3 \\
(Y_i - \bar{Y}) &= (1, -2, -1, 0, 2) && \because \text{subtracting } \bar{Y} = 3 \\
\sum (X_i - \bar{X})(Y_i - \bar{Y}) &= -1.5 + 3 + 0 + 0 + 4 && \because \text{multiplying the two lines above entry by entry} \\
&= 5.5 && \because \text{addition} \\
\sum (X_i - \bar{X})^2 &= 2.25 + 2.25 + 0 + 1 + 4 && \because \text{squaring each entry} \\
&= 9.5 && \because \text{addition} \\
\sum (Y_i - \bar{Y})^2 &= 1 + 4 + 1 + 0 + 4 && \because \text{squaring each entry} \\
&= 10 && \because \text{addition} \\
\rho &= \frac{5.5}{\sqrt{9.5 \times 10}} && \because \text{Given 4 with the three sums above} \\
&= \frac{5.5}{\sqrt{95}} && \because \text{multiplication} \\
&\approx 0.564 && \because \sqrt{95} = 9.7468\ldots, \text{ division, digits after the 3rd d.p. dropped}
\end{aligned}
$$

$\therefore$ Rounding the events to whole epochs drops ρ from 0.70 to 0.564. Rounding changes the answer, which is why ties matter.

$$\boxed{\rho = 0.564 \text{ (3 d.p.)}}$$

Self-check: Which one would seed 4 distort, and why? (Pearson's C on raw values: $G_4 = 23747$ is far above the others, so it dominates the sums in **N41**.)

Likely viva question: "Your Pearson values look strong; why don't you trust them?" (In L2, the final weight norm against grok gives $C = -0.55$ with all seeds and $+0.96$ without seed 4: one seed flips the sign.)

5.3 Only 21 possible values with 5 seeds

Definition: With $n = 5$ and no ties, $\rho = 1 - \sum d_i^2/20$, and $\sum d_i^2$ is an even number from 0 to 40. So ρ can only be $-1, -0.9, \ldots, 0.9, 1$: 21 values, in steps of 0.1.

Why it matters: $\rho \geq 0.9$ means "perfect order, or exactly one adjacent swap" (**N43**). It also explains the coded cutoff 0.8999999999999998 (**N45**): it keeps a ρ of exactly 0.9 from failing through floating-point rounding. The criterion-validation record in `context.md` says ρ has only 11 values in steps of 0.2; that's wrong, and the list it gives leaves out 0.9.

Worked example: The derivation finds the formula for $n = 5$, why $\sum d_i^2$ is even, its largest value, and the ρ of one adjacent swap.

Derivation (H.K. Dass layout): the possible values of ρ with five seeds

Given:

1. $\rho = 1 - 6\sum d_i^2/\big(n(n^2-1)\big)$, no ties (**N42**).
2. $n = 5$, and each list of ranks is $1, 2, 3, 4, 5$ in some order (**N40**).
3. $d_i = R(G_i) - R(P_i)$ is an integer (**N42**).

Let:

1. D: the sum of squared rank differences, $D = \sum_i d_i^2$. Integer ≥ 0. Role: the only thing ρ depends on.
2. $D_{\max}$: the largest possible D. Integer.
3. ρ_{swap}: the ρ when exactly one adjacent pair is swapped. Scalar.

To find: The formula for ρ at $n = 5$, the possible D, and ρ_{swap}.

Solution:

$$\rho = 1 - \frac{6D}{5(5^2-1)} \qquad \because \text{Given 1 with } n=5 \text{ (Given 2) and Let 1}$$

$$= 1 - \frac{6D}{5 \times 24} \qquad \because 5^2 - 1 = 24$$

$$= 1 - \frac{6D}{120} \qquad \because 5 \times 24 = 120$$

$$= 1 - \frac{D}{20} \qquad \because \text{dividing top and bottom by 6}$$

$$\sum_i d_i = \sum_i R(G_i) - \sum_i R(P_i) \qquad \because \text{Given 3, summed over the 5 seeds}$$

$$= 15 - 15 \qquad \because \text{each rank list adds up to } 1+2+3+4+5 = 15 \text{ (Given 2)}$$

$$= 0 \qquad \because \text{subtraction}$$

$$d_i^2 - d_i = d_i(d_i - 1) \qquad \because \text{factoring out } d_i$$

$$D - \sum_i d_i = \sum_i d_i(d_i - 1) \qquad \because \text{summing the line above over the 5 seeds}$$

$$D - 0 = \sum_i d_i(d_i - 1) \qquad \because \textstyle\sum_i d_i = 0\text{, shown above}$$

$$
\begin{aligned}
D &= \text{an even number} && \because\ d_i(d_i-1) \text{ is a product of two consecutive integers, so even} \\
D_{\max} &= 4^2+2^2+0^2+(-2)^2+(-4)^2 && \because\ \text{fully reversed order, } d=(4,2,0,-2,-4) \\
&= 16+4+0+4+16 && \because\ \text{squaring} \\
&= 40 && \because\ \text{addition} \\
\#\{0,2,4,\ldots,40\} &= \frac{40}{2}+1 && \because\ \text{even numbers from 0 to 40} \\
&= 21 && \because\ \text{arithmetic} \\
\rho_{\text{swap}} &= 1-\frac{1^2+(-1)^2}{20} && \because\ \text{one adjacent swap: two seeds have } d_i=\pm1\text{, the rest 0} \\
&= 1-\frac{2}{20} && \because\ 1^2=(-1)^2=1\text{, addition} \\
&= 0.9 && \because\ 2/20=0.1\text{, subtraction}
\end{aligned}
$$

$\therefore$ With five seeds $\rho = 1 - D/20$ with D even from 0 to 40; listing all 120 orderings shows every even value occurs, so ρ has 21 possible values. One adjacent swap gives exactly 0.9; two separate adjacent swaps give $D = 4$ and $\rho = 0.8$, a fail.

$$\boxed{\rho = 1 - \frac{D}{20}, \quad 21 \text{ values}, \quad \rho_{\text{swap}} = 0.9}$$

Self-check: Can $\rho = 0.85$ happen with 5 seeds and no ties? (No: it would need $D = 3$, which is odd.)

Likely viva question: "Isn't the ρ test very coarse with five seeds?"

5.4 Significance with small n

Definition: Significance (**N44**) asks whether a correlation could easily appear by chance, that is, if all $n!$ orderings of the events were equally likely. The p-value is the probability, by chance, of a ρ at least as large as the one observed.

Why it matters: With $n = 5$, only $\rho = 1$, and at the edge $\rho = 0.9$, are unlikely by chance. $\rho = 0.70$ isn't significant.

Worked example: The derivation computes the chance of $\rho = 1$ and of $\rho \geq 0.9$.

Derivation (H.K. Dass layout): chance probabilities of $\rho = 1$ and $\rho \geq 0.9$

Given:

1. $n = 5$; by chance, all $n!$ orderings are equally likely (**N44**).
2. $\rho = 1$ only for the one ordering with $D = 0$; $\rho = 0.9$ only for orderings with exactly one adjacent swap (Section 5.3).
3. With 5 positions there are 4 adjacent pairs: (1,2), (2,3), (3,4), (4,5).

Let:

1. $n!$: the number of orderings of 5 seeds. Integer.
2. $\Pr(\rho = 1)$: chance probability of $\rho = 1$. Scalar in $[0, 1]$.
3. $\Pr(\rho \geq 0.9)$: chance probability of $\rho \geq 0.9$, the p-value of $\rho = 0.9$. Scalar in $[0, 1]$.

To find: $\Pr(\rho = 1)$ and $\Pr(\rho \geq 0.9)$.

Solution:

$$
\begin{aligned}
n! &= 5 \times 4 \times 3 \times 2 \times 1 && \because \text{ definition of } n! \text{ with } n = 5 \\
&= 120 && \because \text{ multiplication} \\
\Pr(\rho = 1) &= \frac{1}{120} && \because \text{ one ordering out of 120 (Given 1, 2)} \\
\Pr(\rho \geq 0.9) &= \frac{1 + 4}{120} && \because \text{ the perfect order plus 4 single adjacent swaps (Given 2, 3)} \\
&= \frac{5}{120} && \because \text{ addition} \\
&= \frac{1}{24} && \because \text{ dividing top and bottom by 5} \\
&\approx 0.042 && \because \text{ division, digits after the 3rd d.p. dropped}
\end{aligned}
$$

$\therefore$ A random event list would reach $\rho = 1$ once in 120 tries, and $\rho \geq 0.9$ about once in 24.

$$\boxed{\Pr(\rho = 1) = \tfrac{1}{120}, \quad \Pr(\rho \geq 0.9) = \tfrac{1}{24}}$$

Self-check: Why does your protocol use a threshold rather than a p-value? (With 120 orderings the p-values are coarse too; a fixed threshold is simpler to pre-register. By the same count, 14 of the 120 orderings give $\rho \geq 0.7$, a p-value of about 0.12.)

Likely viva question: "How confident can you be in a PASS from five points?"

Module 6

The frozen protocol and the PASS rule

6.1 The frozen protocol

Definition: The frozen protocol (**N46**) is the fixed set of rules used to score every predictor: the config `nanda_unified.yaml`, $p = 113$, 5 seeds, grok threshold 0.9, and the PASS thresholds of Section 6.2. It's written down and not changed while predictors are scored.

Why it matters: It makes all predictors comparable, and it stops thresholds being tuned to make a favourite predictor pass. It was written on 29 September 2026, after L2 was closed-negative on 26 September (**N47**). That order is itself an attack point.

Worked example: The verdict function `evaluate_frozen_protocol` in `analyze_nanda_unified.py` (**A9**) applies the three conditions of Section 6.2. There's no calculation here.

Self-check: Which predictors were scored by this function and which were only recomputed later? (L2 was left out of the scored list on 2 October 2026, because no single L2 rule was named as official; its frozen-protocol verdict hasn't been saved to a results file.)

Likely viva question: "Was the protocol fixed before you saw any results?"

6.2 The PASS rule

Definition: A predictor passes (**N45**) only if all three hold:

$$\text{lead_frac} \geq 0.8, \qquad \rho \geq 0.9, \qquad 0.05 \leq \bar{r} \leq 0.9.$$

Otherwise its verdict is FAIL.

Why it matters: Each condition blocks a different failure: lead_frac blocks firing late, ρ blocks firing on a fixed clock, and the upper limit on $\bar{r}$ blocks firing absurdly early.

Worked example: L2 crossover: lead_frac $= 1.0$ (passes, Section 4.3), $\rho = 0.70$ (fails, Section 5.1), $\bar{r} = 0.989$ (fails, Section 4.4), so FAIL. The derivation scores the L2 MA-of-MA zero crossing (**N33**).

Derivation (H.K. Dass layout): the verdict of the L2 MA-of-MA zero crossing

Given:

1. $(G_0, \ldots, G_4) = (12987, 6921, 9510, 11019, 23747)$ and $(R(G_0), \ldots, R(G_4)) = (4, 1, 2, 3, 5)$ (**N27**, Section 5.1).
2. $(P_0, \ldots, P_4) = (1802, 1767, 1814, 1811, 1801)$, all five fire (**N33**).
3. Every P_i is below 1815 and every $G_i - 100$ is at least $6921 - 100 = 6821$, so all five seeds lead (**N37**).

4. $\rho = 1 - D/20$ for $n = 5$ with no ties (Section 5.3).
5. $\bar{r} = 1 - \frac{1}{5}\sum_{i=0}^{4} P_i/G_i$ when $n = 5$ (Section 4.4).
6. PASS rule (**N45**).

Let:

1. $R(P_i)$: rank of P_i among the five events. Integer from 1 to 5. $R(P)$ is the list $(R(P_0), \ldots, R(P_4))$.
2. $d_i = R(G_i) - R(P_i)$: rank difference of seed i. Integer. d is the list $(d_0, \ldots, d_4)$.
3. $D = \sum d_i^2$. Integer.
4. lead_frac, ρ, $\bar{r}$: the three PASS quantities of this rule. Scalars.
5. S: the sum $\sum_{i=0}^{4} P_i/G_i$ for this rule. Scalar, positive.

To find: The three PASS quantities and the verdict.

Solution:

$$\begin{aligned}
\text{lead_frac} &= \frac{5}{5} && \because \text{ Given 3, } n = 5 \\
&= 1.0 && \because \text{ division} \\
\text{sorted } P &: 1767 < 1801 < 1802 < 1811 < 1814 && \because \text{ sorting Given 2: seeds 1, 4, 0, 3, 2} \\
R(P) &= (3, 1, 5, 4, 2) && \because \text{ position of each seed in the line above} \\
d &= (4-3,\ 1-1,\ 2-5,\ 3-4,\ 5-2) && \because \text{ Let 2 with Given 1} \\
&= (1, 0, -3, -1, 3) && \because \text{ subtraction} \\
D &= 1^2 + 0^2 + (-3)^2 + (-1)^2 + 3^2 && \because \text{ Let 3} \\
&= 1 + 0 + 9 + 1 + 9 && \because \text{ squaring} \\
&= 20 && \because \text{ addition} \\
\rho &= 1 - \frac{20}{20} && \because \text{ Given 4} \\
&= 0 && \because 20/20 = 1\text{, subtraction} \\
\bar{r} &= 1 - \frac{S}{5} && \because \text{ Given 5 and Let 5} \\
S &= \tfrac{1802}{12987} + \tfrac{1767}{6921} + \tfrac{1814}{9510} + \tfrac{1811}{11019} + \tfrac{1801}{23747} && \because \text{ Let 5 with Given 1 and 2} \\
&\approx 0.13875 + 0.25531 + 0.19075 + 0.16435 + 0.07584 && \because \text{ each fraction rounded to 5 d.p.; later digits dropped} \\
&= 0.82500 && \because \text{ addition} \\
\bar{r} &= 1 - \frac{0.825}{5} && \because \text{ substitution of } S \text{ into the line } \bar{r} = 1 - S/5 \\
&= 1 - 0.165 && \because \text{ division by 5} \\
&= 0.835 && \because \text{ subtraction}
\end{aligned}$$

$\therefore$ The MA-of-MA crossing has lead_frac $= 1.0$ (passes), $\rho = 0$ (fails) and $\bar{r} = 0.835$ (passes). One failed condition is enough, so the verdict is FAIL. It fires at about epoch 1800 in every seed whatever G_i is, which is a fixed clock.

$$\boxed{\text{lead_frac} = 1.0,\ \rho = 0,\ \bar{r} = 0.835 :\ \text{FAIL}}$$

Self-check: Build a made-up event list that passes all three for your five G_i. (Section 6.3 does this with $P_i = 0.5\,G_i$.)

Likely viva question: "Justify each threshold. Where do 0.8, 0.9, 0.05 and 0.9 come from?"

6.3 Pre-registration and criterion validation

Definition: Pre-registration (**N48**) is writing down your test before running it. Criterion validation is checking that the test passes a good synthetic predictor and fails a bad one, where a synthetic predictor is a made-up list of event epochs built from your G_i.

Why it matters: Your `context.md` records a calibration of the ρ cutoff. It used this run's own grok epochs, after L2's results were already known, so an examiner can call it post-hoc.

Worked example: The derivation scores the synthetic "perfect" predictor $P_i = 0.5\,G_i$.

Derivation (H.K. Dass layout): the synthetic predictor $P_i = 0.5\,G_i$

Given:

1. $(G_0, \ldots, G_4) = (12987, 6921, 9510, 11019, 23747)$, all different (**N27**).
2. Synthetic events $P_i = 0.5\,G_i$ for every seed, so all five fire.
3. Lead: $P_i < G_i - 100$ (**N37**); $r_i = (G_i - P_i)/G_i$ (**N39**).
4. PASS rule (**N45**).

Let:

1. P_i: synthetic event epoch of seed i. Scalar epoch.
2. r_i: its relative lead. Scalar.
3. lead_frac, ρ, $\bar{r}$: the three PASS quantities. Scalars.

To find: The three PASS quantities and the verdict.

Solution:

$$
\begin{aligned}
G_i - P_i &= G_i - 0.5\,G_i && \because \text{Given 2} \\
&= 0.5\,G_i && \because \text{subtraction} \\
&\geq 0.5 \times 6921 && \because \text{the smallest } G_i \text{ is 6921 (Given 1)} \\
&= 3460.5 && \because \text{multiplication} \\
&> 100 && \because \text{comparison, so } P_i < G_i - 100 \text{ in every seed} \\
\text{lead_frac} &= \frac{5}{5} && \because \text{all five seeds lead (Given 3, line above)} \\
&= 1.0 && \because \text{division} \\
R(P_i) &= R(G_i) && \because P_i = 0.5\,G_i \text{ keeps the order of the } G_i
\end{aligned}
$$

$$
\begin{aligned}
d_i &= 0 && \because\ d_i = R(G_i) - R(P_i) \\
\rho &= 1 - \frac{0}{20} && \because\ \text{Section 5.3 with } D = 0 \\
&= 1 && \because\ \text{arithmetic} \\
r_i &= \frac{0.5\, G_i}{G_i} && \because\ \text{Given 3 and the second line} \\
&= 0.5 && \because\ G_i / G_i = 1 \\
\bar{r} &= \frac{5 \times 0.5}{5} && \because\ \text{all five } r_i \text{ equal 0.5} \\
&= 0.5 && \because\ \text{division}
\end{aligned}
$$

$\therefore$ The perfect synthetic predictor gets lead_frac $= 1.0$, $\rho = 1$ and $\bar{r} = 0.5$, so it must PASS. A synthetic event at a fixed epoch, e.g. $P_i = 1800$ in every seed, has all events tied, so ρ is undefined (Section 5.1) and can't reach 0.9; it must FAIL.

$$\boxed{\text{lead_frac} = 1.0,\ \rho = 1,\ \bar{r} = 0.5 :\ \text{PASS}}$$

Self-check: What would a synthetic control that should fail look like? (The fixed $P_i = 1800$ above; or an event placed after grok, $P_i = G_i + 100$, which has no leads.)

Likely viva question: "How do you know your criterion isn't too strict to pass anything?" (The perfect synthetic predictor passes it.)

Module 7

Research honesty

7.1 Hindsight (look-ahead) bias

Definition: Look-ahead bias (**N49**) is using information from later epochs, such as the final value y_{final} of a signal at epoch 40,000 (**N28**), to decide when an earlier event happened.

Why it matters: Two readings pass only because they use the run's final value: the L2 halfway reading (**N33**) and the Dropout 10% rule (**N34**). A live monitor couldn't run them during training. Under the frozen rule the halfway reading gets lead_frac $= 0.8$, $\rho = 0.9$ (exactly the cutoff) and $\bar{r} = 0.162$, so it would PASS if it were scored. You've recorded it as a hindsight check, not a benchmark result.

Worked example: The derivation rewrites the halfway target to show that it depends on y_{final}.

Derivation (H.K. Dass layout): the halfway target needs the final value

Given:

1. $y_{\text{target}} = y_{\text{peak}} - 0.5(y_{\text{peak}} - y_{\text{final}})$, with $y_t = S_t$ (**N33**).
2. y_{peak} is the largest y_t over the run; y_{final} is y_t at $t = 40{,}000$ (**N28**).
3. The halfway event lies between epochs 1038 and 40,000 (it comes after the peak, **N33**).

Let:

1. y_{peak}: the peak of S_t. Scalar ≥ 0, known once the peak has passed.
2. y_{final}: S_t at epoch 40,000. Scalar ≥ 0, known only at the end of the run.
3. y_{target}: the level the halfway reading waits for. Scalar.

To find: y_{target} written as a sum of y_{peak} and y_{final} terms.

Solution:

$$
\begin{aligned}
y_{\text{target}} &= y_{\text{peak}} - 0.5(y_{\text{peak}} - y_{\text{final}}) && \because \text{ Given 1} \\
&= y_{\text{peak}} - 0.5\, y_{\text{peak}} + 0.5\, y_{\text{final}} && \because \text{ multiplying out: } -0.5(x-y) = -0.5x + 0.5y \\
&= (1 - 0.5)\, y_{\text{peak}} + 0.5\, y_{\text{final}} && \because \text{ factoring } y_{\text{peak}} \text{ out of the first two terms} \\
&= 0.5\, y_{\text{peak}} + 0.5\, y_{\text{final}} && \because \ 1 - 0.5 = 0.5
\end{aligned}
$$

$\therefore$ The target is half the peak plus half the final value. It has a non-zero share of y_{final}, which is only known at epoch 40,000, while the event it defines comes earlier (Given 3). So the rule looks ahead.

$$\boxed{y_{\text{target}} = 0.5\, y_{\text{peak}} + 0.5\, y_{\text{final}}}$$

Self-check: How would you test whether a rule is look-ahead free? (Cut the curve at epoch t, rerun the rule, and check that the event before t doesn't move.)

Likely viva question: "Could this signal be used during training, before you know the outcome?"

7.2 Causal vs centred moving average

Definition: A causal moving average at epoch t (**N29**) uses only points at or before t. A centred one uses points on both sides of t.

Why it matters: `uniform_filter1d` is centred (with the edges padded by copying), and both L2 rules use it, so each smoothed value includes later epochs. Any event found on such a curve is slightly look-ahead. Your viva notes call the MA crossover an "ex-ante trigger" (one that uses only the past); strictly it isn't.

Worked example: The derivation writes both averages for a window of 3 points.

Derivation (H.K. Dass layout): causal and centred averages, window 3

Given:

1. A signal y_t, one scalar per point t (**N28**).
2. Window $w = 3$; causal uses points $\leq t$, centred uses both sides (**N29**).

Let:

1. y_t: the signal at point t. Scalar.
2. $\bar{y}_t^{\text{causal}}$: the causal average at t. Scalar. Uses y_{t-2}, y_{t-1}, y_t.
3. $\bar{y}_t^{\text{centred}}$: the centred average at t. Scalar. Uses y_{t-1}, y_t, y_{t+1}.

To find: Both averages, and which needs a later point.

Solution:

$$\bar{y}_t^{\text{causal}} = \frac{y_{t-2} + y_{t-1} + y_t}{3} \qquad \because \text{ the 3 points ending at } t \text{ (Given 2)}$$

$$\bar{y}_t^{\text{centred}} = \frac{y_{t-1} + y_t + y_{t+1}}{3} \qquad \because \text{ one point each side of } t \text{ (Given 2)}$$

$\therefore$ The centred average at t contains y_{t+1}, a point from the future; the causal one doesn't.

$$\boxed{\bar{y}_t^{\text{centred}} \text{ needs } y_{t+1};\ \bar{y}_t^{\text{causal}} \text{ doesn't}}$$

Self-check: Which one shifts events later? (The causal one: it only reacts after the change has happened, so its events come later than the centred version's.)

Likely viva question: "Is your smoothing causal?" (No: `uniform_filter1d` is centred.)

7.3 Researcher degrees of freedom

Definition: Researcher degrees of freedom (**N50**) are choices of windows, thresholds or `skip_epochs` (**N35**) made after looking at the results.

Why it matters: Several rules were changed after looking at curves, so an examiner can say they were fitted to the data.

Worked example: L2's `skip_epochs` went from 200 to 100 after you inspected the curves, and the crossover then fires just after 100. The fast and slow averages already cross upwards several times in the first 100 epochs (in seed 0 at about epochs 2, 3 and 42); the rule reports 113 to 137 only because it ignores everything before epoch 100. There's no calculation here.

Self-check: Which settings in your config say "not from a paper"? (Check each window and skip value in `nanda_unified.yaml` for a citation.)

Likely viva question: "How many rules did you try before this one?"

7.4 Sensitivity analysis

Definition: Sensitivity analysis (**N51**) is changing one setting a little and checking whether the verdict changes.

Why it matters: None of the window sizes or margins were varied. Near-misses show how fragile some verdicts are.

Worked example: The L2 peak event (**N33**) gives $\bar{r} = 0.904$ and $\rho = 0.9$. The derivation measures how far it misses.

Derivation (H.K. Dass layout): how far the L2 peak event misses

Given:

1. L2 peak event: $\bar{r} = 0.904$ (saved L2 results).
2. Upper limit on $\bar{r}$: 0.9 (**N45**).

Let:

1. $\bar{r}$: the mean relative lead of the peak event. Scalar.
2. δ: the miss, $\delta = \bar{r} - 0.9$. Scalar; positive means above the limit.

To find: δ.

Solution:

$$\begin{aligned} \delta &= \bar{r} - 0.9 && \because \text{ definition of } \delta \text{ (Let 2)} \\ &= 0.904 - 0.9 && \because \text{ substitution of Given 1} \\ &= 0.004 && \because \text{ subtraction} \end{aligned}$$

$\therefore$ The peak event fails only by 0.004. A slightly higher upper limit would turn it into a PASS, even though it fires between epochs 1038 and 1061 in every seed, only 23

epochs apart.

$$\boxed{\delta = 0.004}$$

Self-check: Which threshold would you vary first? (The upper limit 0.9 on $\bar{r}$, since the peak event sits 0.004 above it.)

Likely viva question: "Would your verdict survive a different window size?"

7.5 Baselines, controls and noise floor

Definition: A control run (**N52**) is one where grokking doesn't happen, or a setting is switched off. The noise floor is how much a signal wobbles for no reason.

Why it matters: Without them you can't say a change in a signal is because of grokking. For L2 the noise floor is computed and saved but isn't used in any decision, and it's measured over epochs before 90, which are mostly interpolated grid points (**N32**).

Worked example: A run with weight decay $\lambda = 0$ (**N19**), which often doesn't grok, would show what S_t does without grokking. There's no calculation here.

Self-check: How would you estimate the noise floor from the saved curves?

Likely viva question: "Where's your control condition?"

7.6 Negative results and reproducibility

Definition: A negative result (**N53**) is a clear "this doesn't work". Reproducibility means someone else can rerun your work and get the same numbers.

Why it matters: Seven of your eight predictors are closed-negative (**N47**), so the thesis's value is in showing the tests were fair and repeatable: fixed seeds, saved data, frozen thresholds.

Worked example: Recomputing from the saved accuracy curves gives the same G_i, and rebuilding the split with the same seed reproduces the saved accuracies. The L2 events recomputed from the saved files match `summary.json`. There's no calculation here.

Self-check: What makes a negative result publishable rather than just a failure?

Likely viva question: "If almost nothing passes, what is the contribution of the thesis?"

Recommended reading

[1] Power, A., Burda, Y., Edwards, H., Babuschkin, I., Misra, V. (2022). *Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets.* arXiv:2201.02177. (Modules 1–2.)

[2] Nanda, N., Chan, L., Lieberum, T., Smith, J., Steinhardt, J. (2023). *Progress Measures for Grokking via Mechanistic Interpretability.* ICLR 2023. (Modules 1–2; your setup.)

[3] Liu, Z., Michaud, E. J., Tegmark, M. (2023). *Omnigrok: Grokking Beyond Algorithmic Data.* ICLR 2023. (Weight norm and weight decay.)

[4] Loshchilov, I., Hutter, F. (2019). *Decoupled Weight Decay Regularization.* ICLR 2019. (AdamW.)

[5] Spearman, C. (1904), the original paper, or any introductory statistics chapter on Spearman rank correlation. (Module 5.)

[6] Nosek, B. A. et al. (2018). *The Preregistration Revolution.* PNAS 115(11). (Modules 6–7.)